

使用 Draco 幾何壓縮功能最佳化 3D 資料

來源：<https://codelabs.developers.google.com/codelabs/draco-3d?hl=zh-tw>

步驟數：8

目錄

1. [1. 總覽](#)
2. [2. 開始設定](#)
3. [3. 建構編碼器](#)
4. [4. 編碼第一個 3D 素材資源](#)
5. [5. 在瀏覽器中解碼 Draco 檔案](#)
6. [6. 使用 three.js 算繪 Draco 檔案](#)
7. [7. 嘗試使用不同的編碼參數](#)
8. [8. 恭喜](#)

1. 總覽

3D 圖像是許多應用程式的基本要素，包括遊戲、設計和資料視覺化。隨著繪圖處理器和創作工具不斷改良，更大更複雜的 3D 模型將成為常態，並協助推動沉浸式虛擬實境 (VR) 和擴增實境 (AR) 的新應用。由於模型複雜度提高，儲存空間和頻寬需求也必須跟上 3D 資料爆炸性成長的腳步。

使用 Draco 後，應用程式可大幅縮減 3D 圖像的大小，同時維持視覺保真度。對使用者來說，這代表應用程式下載速度更快、瀏覽器中的 3D 圖像載入速度更快，而且傳輸 VR 和 AR 場景時，頻寬用量僅為一小部分，因此能快速算繪並呈現絕佳效果。

什麼是 Draco？

Draco 是一個程式庫，用於壓縮及解壓縮 3D 幾何[網格](#)和[點雲](#)。這項技術旨在提升 3D 圖像的儲存和傳輸效率。

Draco 的設計和建構宗旨是提升壓縮效率和速度。程式碼支援壓縮點、連線資訊、紋理座標、顏色資訊、法線，以及與幾何圖形相關聯的任何其他一般屬性。Draco 以 C++ 原始碼的形式發布，可用於壓縮 3D 圖像，以及用於編碼資料的 C++ 和 JavaScript 解碼器。

學習目標

- 如何使用 Draco 壓縮 3D 模型
- 如何使用不同的壓縮模型，以及這些模型對模型品質和大小的影響
- 如何在網頁上查看 3D 模型

軟硬體需求

- [Chrome](#) 近期版本
 - [範例程式碼](#)
 - 文字編輯器
 - Python
 - [cmake](#)
-

步驟 2 · 約 0 分鐘

2. 開始設定

使用下列指令複製 GitHub 存放區：



```
git clone https://github.com/google/draco
```

前往 Draco 根目錄。



```
cd draco
```

步驟 3 · 約 0 分鐘

3. 建構編碼器

如要開始使用 Draco 編碼和解碼，請先建構應用程式。

建構編碼器

- 從要產生建構檔案的目錄執行 `cmake`，並將 Draco 存放區的路徑傳遞給 `cmake`。

建議您在 Draco 根目錄下建立建構目錄。

注意：您無法在 Draco 根目錄外執行 `cmake`。

```
mkdir build

cd build

cmake ../

make
```

步驟 4 · 約 0 分鐘

4. 編碼第一個 3D 素材資源

draco_encoder 會讀取 OBJ 或 PLY 檔案做為輸入內容，並輸出 Draco 編碼的檔案。我們已納入 Stanford 的 Bunny 網格以供測試。基本指令列如下所示：

```
./draco_encoder -i ../testdata/bun_zipper.ply -o bunny.drc
```

現在可以查看輸出檔案的大小，並與原始 .ply 檔案比較。壓縮檔的大小應遠小於原始檔案。

原始檔案為 2.9 MB，壓縮後的檔案約為 46 kB。大小縮減超過 60 倍。

注意：壓縮後的大小會因壓縮選項而異。

5. 在瀏覽器中解碼 Draco 檔案

此時，我們會先從解碼 Draco 檔案的基本網頁開始。首先，請複製下列程式碼區段並貼到文字編輯器中。

1. 先從基本的 HTML 檔案著手。

```
<!DOCTYPE html>
<html>
<head>
  <title>Codelab - Draco Decoder</title>
```

2. 下列程式碼片段會載入 Draco WASM 解碼器。

建議您一律從 <https://www.gstatic.com/draco/versioned/decoders/1.4.1/> 提取 Draco WASM 解碼器，因為這樣一來，使用者就能從快取中存取 Draco 解碼器。

```
<script src="https://www.gstatic.com/draco/versioned/decoders/1.4.1/draco_wasm_wrapper.js">
  // It is recommended to always pull your Draco JavaScript and WASM decoders
  // from the above URL. Users will benefit from having the Draco decoder in
  // cache as more sites start using the static URL.
</script>
```

3. 接著新增這個函式，建立 Draco 解碼器模組。解碼器模組的建立作業是非同步的，因此您必須等待系統呼叫回呼，才能使用該模組。

```
<script>
  'use strict';

  // The global Draco decoder module.
  let decoderModule = null;

  // Creates the Draco decoder module.
  function createDracoDecoderModule() {
    let dracoDecoderType = {};

    // Callback when the Draco decoder module is fully instantiated. The
    // module parameter is the created Draco decoder module.
    dracoDecoderType['onModuleLoaded'] = function(module) {
      decoderModule = module;

      // Download the Draco encoded file and decode.
      downloadEncodedMesh('bunny.drc');
    };

    DracoDecoderModule(dracoDecoderType);
  }
}
```

4. 新增函式，解碼以 Draco 編碼的網格。

```

// Decode an encoded Draco mesh. byteArray is the encoded mesh as
// an Uint8Array.
function decodeMesh(byteArray) {
    // Create the Draco decoder.
    const decoder = new decoderModule.Decoder();

    // Create a buffer to hold the encoded data.
    const buffer = new decoderModule.DecoderBuffer();
    buffer.Init(byteArray, byteArray.length);

    // Decode the encoded geometry.
    let outputGeometry = new decoderModule.Mesh();
    let decodingStatus = decoder.DecodeBufferToMesh(buffer, outputGeometry);

    alert('Num points = ' + outputGeometry.num_points());

    // You must explicitly delete objects created from the DracoModule
    // or Decoder.
    decoderModule.destroy(outputGeometry);
    decoderModule.destroy(decoder);
    decoderModule.destroy(buffer);
}

```

5. 現在我們已備妥 Draco 解碼函式，請新增函式來下載 Draco 編碼網格。「downloadEncodedMesh」函式會接受要載入的 Draco 檔案參數。在本例中，這會是上一個階段的「bunny.drc」。

```

// Download and decode the Draco encoded geometry.
function downloadEncodedMesh(filename) {
    // Download the encoded file.
    const xhr = new XMLHttpRequest();
    xhr.open("GET", filename, true);
    xhr.responseType = "arraybuffer";
    xhr.onload = function(event) {
        const arrayBuffer = xhr.response;
        if (arrayBuffer) {
            const byteArray = new Uint8Array(arrayBuffer);
            decodeMesh(byteArray);
        }
    };
    xhr.send(null);
}

```

6. 呼叫「createDracoDecoderModule」函式來建立 Draco 解碼器模組，該模組會呼叫「downloadEncodedMesh」函式來下載編碼的 Draco 檔案，並呼叫「decodeMesh」函式來解碼編碼的 Draco 網格。

```
// Create the Draco decoder module.  
createDracoDecoderModule();  
</script>  
</head>  
<body>  
</body>  
</html>
```

7. 將這個檔案儲存為「DracoDecode.html」

8. 啟動 Python 網路伺服器。在終端機中輸入：

```
python -m SimpleHTTPServer
```

9. 在 Chrome 中開啟 localhost:8000/DracoDecode.html。這時應該會顯示警告訊息方塊，指出從模型解碼的點數 (點數 = 34834)。

6. 使用 three.js 算繪 Draco 檔案

我們已瞭解如何使用 WASM 解碼 Draco 檔案，現在要使用熱門的網頁 3D 檢視器 three.js。如同上一個範例，我們將從複製下列程式碼區段並貼到文字編輯器開始。

1. 從基本 HTML 檔案開始

```
<!DOCTYPE html>
<html>
<head>
  <title>Codelab - Draco three.js Render</title>
```

2. 加入程式碼，載入 three.js 和 Draco three.js 載入器。

```
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@v0.162.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@v0.162.0/examples/jsm/"
    }
  }
</script>
```

3. 設定 Draco 解碼器路徑。

```
<script type="module">
  // import three.js and DRACOLoader.
  import * as THREE from 'three';
  import {DRACOLoader} from 'three/addons/loaders/DRACOLoader.js'

  // three.js globals.
  var camera, scene, renderer;

  // Create the Draco loader.
  var dracoLoader = new DRACOLoader();

  // Specify path to a folder containing WASM/JS decoding libraries.
  // It is recommended to always pull your Draco JavaScript and WASM decoders
  // from the below URL. Users will benefit from having the Draco decoder in
  // cache as more sites start using the static URL.
  dracoLoader.setDecoderPath('https://www.gstatic.com/draco/versioned/decoders/1.4.1/');
```

4. 新增 three.js 算繪程式碼。

```

function initThreejs() {
    camera = new THREE.PerspectiveCamera( 35, window.innerWidth / window.innerHeight, 0.1,
15 );
    camera.position.set( 3, 0.25, 3 );

    scene = new THREE.Scene();
    scene.background = new THREE.Color( 0x443333 );
    scene.fog = new THREE.Fog( 0x443333, 1, 4 );

    // Ground
    var plane = new THREE.Mesh(
        new THREE.PlaneGeometry( 8, 8 ),
        new THREE.MeshPhongMaterial( { color: 0x999999, specular: 0x101010 } )
    );
    plane.rotation.x = - Math.PI / 2;
    plane.position.y = 0.03;
    plane.receiveShadow = true;
    scene.add(plane);

    // Lights
    var light = new THREE.HemisphereLight( 0x443333, 0x111122 );
    scene.add( light );

    var light = new THREE.SpotLight();
    light.angle = Math.PI / 16;
    light.penumbra = 0.5;
    light.castShadow = true;
    light.position.set( - 1, 1, 1 );
    scene.add( light );

    // renderer
    renderer = new THREE.WebGLRenderer( { antialias: true } );
    renderer.setPixelRatio( window.devicePixelRatio );
    renderer.setSize( window.innerWidth, window.innerHeight );
    renderer.shadowMap.enabled = true;

    const container = document.getElementById('container');
    container.appendChild( renderer.domElement );

    window.addEventListener( 'resize', onWindowResize, false );
}

function onWindowResize() {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();

    renderer.setSize( window.innerWidth, window.innerHeight );
}

function animate() {
    render();
    requestAnimationFrame( animate );
}

```

```

}

function render() {
  var timer = Date.now() * 0.0003;

  camera.position.x = Math.sin( timer ) * 0.5;
  camera.position.z = Math.cos( timer ) * 0.5;
  camera.lookAt( new THREE.Vector3( 0, 0.1, 0 ) );

  renderer.render( scene, camera );
}

```

5. 新增 Draco 載入和解碼程式碼。

```

function loadDracoMesh(dracoFile) {
  dracoLoader.load(dracoFile, function ( geometry ) {
    geometry.computeVertexNormals();

    var material = new THREE.MeshStandardMaterial( { vertexColors: THREE.VertexColors }
  );
  var mesh = new THREE.Mesh( geometry, material );
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  scene.add( mesh );
} );
}

```

6. 新增程式碼來載入檔案。

```

window.onload = function() {
  initThreejs();
  animate();
  loadDracoMesh( 'bunny.drc' );
}
</script>
</head>
<body>
  <div id="container"></div>
</body>
</html>

```

7. 將這個檔案儲存為「DracoRender.html」

8. 如有需要，請重新啟動網路伺服器。

```
python -m SimpleHTTPServer
```

8. 在 Chrome 中開啟 localhost:8000/DracoRender.html。現在瀏覽器中應該會顯示 Draco 檔案。

7. 嘗試使用不同的編碼參數

Draco 編碼器允許許多不同參數，這些參數會影響壓縮檔案的大小和程式碼的視覺品質。請嘗試在指令列中執行接下來的幾個指令，並查看結果。

1. 下列指令會使用 12 位元 (預設為 11 位元) 量化模型的位置。

```
./draco_encoder -i ../testdata/bun_zipper.ply -o out12.drc -qp 12
```

2. 請注意 out12.drc 的大小，與上一節中的 bunny.drc 檔案相比。使用更多量化位元可能會增加壓縮檔案的大小。

3. 下列指令會使用 6 位元量化模型的位置。

```
./draco_encoder -i ../testdata/bun_zipper.ply -o out6.drc -qp 6
```

4. 請注意 out6.drc 的大小，並與上一節中的 bunny.drc 檔案比較。減少量化位元數可縮減壓縮檔案的大小。

使用過少的位元進行量化，雖然可達到驚人的壓縮比，但可能會嚴重降低模型品質。如果算繪 6.drc，您會發現視覺品質與原始檔案大不相同。

5. 下列參數會影響模型的壓縮層級。使用 cl 旗標，即可將壓縮率從 1 (最低壓縮率) 調整為 10 (最高壓縮率)。

```
./draco_encoder -i ../testdata/bun_zipper.ply -o outLow.drc -cl 1  
  
./draco_encoder -i ../testdata/bun_zipper.ply -o outHigh.drc -cl 10
```

請注意 Draco 編碼器的輸出內容。您可以看到，與節省的位元數相比，以最高壓縮層級壓縮所需的時間會有所取捨。應用程式的正確參數取決於編碼時的時間和大小需求。

步驟 8 · 約 0 分鐘

8. 恭喜

您已完成 Draco 網格壓縮程式碼研究室，並成功探索 Draco 的許多重要功能！

希望您已清楚瞭解 Draco 如何協助縮小 3D 素材資源，並更有效率地透過網路傳輸。如要進一步瞭解 Draco 並取得最新程式庫，請前往 [GitHub](#)。
